Fraud detection in credit card transactions using Quantum-Assisted Restricted Boltzmann Machines

João M. Neto* and Guilherme P. Temporão
NITeQ, Electrical Engineering Department
Pontifícia Universidade Católica do Rio de Janeiro,
Rio de Janeiro RJ 22451-900, Brazil

Gustavo C. Amaral
Quantum Technology Department,
The Netherlands Organization for Applied
Scientific Research, TNO, The Netherlands

arXiv:2512.17660v1 [quant-ph] 19 Dec 2025

Use cases for emerging quantum computing platforms become economically relevant as the efficiency of processing and availability of quantum computers increase. We assess the performance of Restricted Boltzmann Machines (RBM) assisted by quantum computing, running on real quantum hardware and simulators, using a real dataset containing 145 million transactions provided by Stone, a leading Brazilian fintech, for credit card fraud detection. The results suggest that the quantum-assisted RBM method is able to achieve superior performance in most figures of merit in comparison to classical approaches, even using current noisy quantum annealers. Our study paves the way for implementing quantum-assisted RBMs for general fault detection in financial systems.

I. INTRODUCTION:

Quantum computing was proposed by Richard Feynman as a way to use quantum physics to effectively simulate quantum physics [1]. Since then, quantum computing has been evolving and quantum algorithms have been developed, such as Shor's algorithm [2], capable of factoring large integers more efficiently than any supercomputer today.

Alongside algorithms, efforts have been made towards the physical implementation of quantum computers, such as a 3000 qubit ion trap computer [3], photonic quantum computers [4], and superconducting quantum processors operating below the surface code error correction threshold [5]. There are several ways to physically implement quantum computing, such as gate-based quantum computing (GBQC) [6], adiabatic quantum computing (AQC) [7], quantum computation based on anyons [8], and quantum computation based on continuous variables [9]. Since this work involves the resolution of different Quadratic Unconstrained Binary Optimization (QUBO) problems, an approximation of AQC, called quantum annealing (QA), was chosen. It has been shown to be a feasible way to solve said problems [10, 11].

AQC consists of initializing the qubits in a simple Hamiltonian, with a known ground state. This Hamiltonian is then adiabatically evolved to a more complex configuration, which will encode the target problem. The main idea is that, by adiabatically evolving the simple Hamiltonian while the qubits are initialized in its ground state configuration, the system will remain in its ground state. According to the adiabatic theorem [12], the final state of the qubits will correspond to the final state of the Hamiltonian, which corresponds to the solution of the problem.

The aim of this study is to investigate the performance of machine learning algorithms (MLA) trained using quantum annealing and applied to classification problems. In particular, we combine a real world dataset, provided by the Brazilian fintech Stone, and compare the performances of classical and quantum-assisted-MLAs for fraud detection. A famous MLA is the Restricted Boltzmann Machine (RBM), an energy based network [13] whose energy model is mathematically equivalent to QA [14]. By writing the energy function of the RBM as a QUBO problem, it is straightforward to transform it into an Ising Hamiltonian, which can be effectively optimized by QA on D-Wave's quantum computers and simulators [15]. This approach follows a similar framework presented in [15]. Simultaneously, classical training is performed using Persistent Contrastive Divergence (PCD). To assist in the training procedure and the creation of RBMs, the Julia programming language module `QARBoM.jl` [16] is deployed.

The paper is organized as follows. Section II provides a deeper dive into RBMs, QA, and Quantum Sampling (QS); section III focuses on the experimental implementation; section IV on the results; and section V on the final conclusions and future work directions

II. METHODS

A. Restricted Boltzmann Machines:

A Restricted Boltzmann Machine (RBM) is a probabilistic generative machine learning model capable of handling supervised and unsupervised data, first introduced by Hinton et al [13]. Since its training depends

* e-mail:joao@niteqpuc.com.

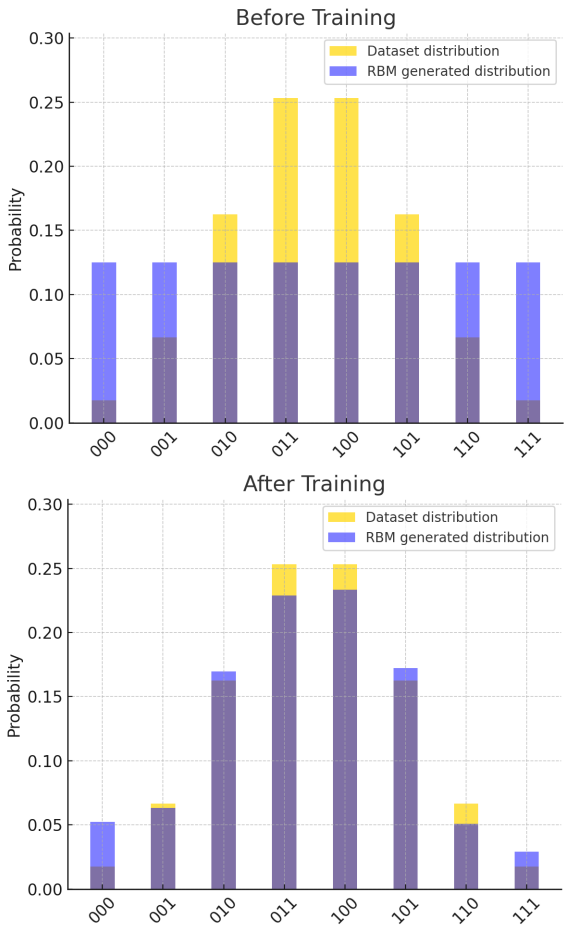

FIG. 1. Representation of the probability distribution p($\mathbf{v}$) before (top) and after (bottom) training the RBM. Probability distribution of the dataset is shown in yellow, while the distribution of the data generated by the model can be seen in blue.

on an intractable partition function, approximations are needed to compute the parameter updates the procedure, as will be discussed in sec. II B. RBMs were developed as probabilistic generative models capable of learning and replicating the probability distribution of a dataset. Nevertheless, following [17], they can also work as classifiers, known as Restricted Boltzmann Machine Classifiers (RBMCs).

An RBM consists of two layers: a visible layer ($\mathbf{v}$), where the visible units $\mathbf{v}_i$ lie, and a hidden layer ($\mathbf{h}$) with hidden units $\mathbf{h}_i$. The main goal when training an RBM is the minimization of its energy function, which leads to the replication of the probability distribution of the dataset [18]. Adjusting the values of the biases and weights unlocks the possibility to adjust the probability distribution of the data generated by the model to match the probability distribution of the dataset, as pictorially displayed in Figure 1.

The energy function of an RBM can be represented as follows [13]:

$$E(\mathbf{v},\mathbf{h}) = -\sum_{i,j} v_i W_{ij} h_j - \sum_i c_i v_i - \sum_j b_j h_j, \qquad (1)$$

where c_i represents the bias terms for the visible units and b_j the bias terms in relation to the visible units. With the energy function of Eq. 1, we can define the joint probability distribution of $\mathbf{v}$ and $\mathbf{h}$ as follows:

$$p(\mathbf{v},\mathbf{h}) = \frac{1}{Z} e^{-E(\mathbf{v},\mathbf{h})}, \qquad (2)$$

where

$$Z = \sum_v \sum_h e^{-E(\mathbf{v},\mathbf{h})} \qquad (3)$$

is a partition function. Although fundamental for the notion of probability, as it represents the normalization of the joint probability distribution, Z depends on the summation of every single configuration of $\mathbf{v}$ and $\mathbf{h}$. This makes the function calculation intractable, i.e., it can be unfeasible to directly compute the joint probability distribution for large problems in classical computers.

The original RBM deals with binary data, making it incompatible with the characteristics of the dataset used in this work. An important extension of RBMs is the Gaussian Restricted Boltzmann Machine (GRBM) [19], suitable for handling continuous-valued data. The marginal probability distribution of the visible layers is altered: instead of a Bernoulli distribution, we now have a continuous Gaussian distribution. The training methodology, discussed in the next Section, is analogous; the details that make it compatible with GRBMs will be highlighted when necessary.

B. Training Restricted Boltzmann Machines:

Training RBMs is a non-trivial task; recall that our goal is to minimize the energy function, which is equivalent to attributing high probability to examples generated akin to reality; in this case, the training dataset. In order to do so, one must first define a loss function to update the parameters. This loss function is generally defined as the log-likelihood of the probability distribution [13], which is given by:

$$L(\mathbf{V}) = -\sum_{i=0}^{V} \log(p(\mathbf{v}^{(i)})), \qquad (4)$$

By using stochastic gradient descent (SGD) [20], parameters of the RBM can be updated, making sure that the probability distribution of the data generated by the RBM is as close as it can get to the dataset. For the update of said parameters, the following useful mathematical relations are used [21]:

$$\frac{\partial log(p(\mathbf{v}^{(i)}))}{\partial \mathbf{W}} = \mathbf{h}_{\text{data}} \mathbf{v}_{\text{data}}^T - \mathbf{h}_{\text{model}} \mathbf{v}_{\text{model}}^T, \qquad (5)$$

$$\frac{\partial log(p(\mathbf{v}^{(i)}))}{\partial \mathbf{b}} = \mathbf{h}_{\text{data}} - \mathbf{h}_{\text{model}}, \tag{6}$$

$$\frac{\partial log(p(\mathbf{v}^{(i)}))}{\partial \mathbf{b}} = \mathbf{v}_{\text{data}} - \mathbf{v}_{\text{model}}, \tag{7}$$

The calculation of the gradient in relation to the positive phase in Eqs. 5-7 is trivial since the probability distribution of the data is given: the expected value of h_{data} can be estimated by sampling its conditional probability in relation to $\mathbf{v}$. The negative phase, on the other hand, is not so trivial: for the calculation of the expected value, the joint probability of $\mathbf{v}_{model}$ and $\mathbf{h}_{model}$ would need to be sampled, which is intractable [13]. Fortunately, one can circumvent this issue by calculating an approximation of the expected values using the *Persistence Contrastive Divergence* (PCD) method [22] via the so-called – classically inefficient – Gibbs Sampling algorithm [23].

The training of a GRBM is done the same way as the RBM. The log-likelihood is the loss function, and the gradients can be updated the same way as before. The only difference is how the conditional probability distribution, necessary to calculate the gradients, is calculated [24]. By assuming that $\mathbf{v}$ is a continuous Gaussian distribution with zero mean and unitary standard deviation, we arrive at the following conditional probability distribution:

$$p(\mathbf{v} \mid \mathbf{h}) = \mathcal{N}(\mathbf{c} + \mathbf{W}^{\boldsymbol{T}}\mathbf{h}, \mathbf{I}), \tag{8}$$

where $\mathbf{I}$ represents the identity matrix.

C. Restricted Boltzmann Machines as a QUBO problem and Ising Model Formulation:

Quadratic Unconstrained Binary Optimization (QUBO) problems [25] aim at minimizing the following objective function:

$$f(x) = \sum_i Q_{i,i}x_{i,i} + \sum_{i<j} Q_{ij}x_ix_j, \tag{9}$$

where x represents binary variables and $Q_{i,j}$ are coefficients of the symmetric matrix $\mathbf{Q}$, which determine linear and quadratic weights. With the objective function defined, we now need to write the energy function of the RBM as a QUBO problem, with slight modifications, as follows. First the visible, label, and hidden layers will all be concatenated in a single vector $\mathbf{x} = (\mathbf{v}, \mathbf{y}, \mathbf{h})$. Second, an ideal $\mathbf{Q}$ matrix must be found with the property that, when multiplied on the left by $\mathbf{x}$ and on the right by $\mathbf{x}^T$ it gives the correct loss function for Gaussian-Bernoulli RBMs [21]. In summary, to construct the $\mathbf{Q}$ matrix, one should follow the following rules:

1. $Q_{i,i}$ represents the biases.
2. $Q_{i,j}$, represents the weights for $i \neq j$.
3. $Q_{i,j} = 0$, if i and j are the same type of unit.
4. $Q_{i,j} = Q_{j,i}$.

It is important to note that, when writing the energy function of the RBM as a QUBO problem, if the visible units are Gaussian, then a binary expansion is needed in order to still maintain the overall structure of the problem. This increases the number of variables and will eventually become a bottleneck when implementing the RBM in a quantum computer, which is the focus of this work. Although an issue, it also represents a necessary step in order to achieve the goal of assisting RBM training with a quantum computer.

With the $\mathbf{Q}$ and $\mathbf{x}$, one can finally write the RBM as a QUBO problem. By making used of a simple transformation, presented below, it is possible to convert the QUBO to an Ising Hamiltonian [15]. This enables one to perform the optimization efficiently when assisted by an adiabatic quantum computer such as D-Wave's system [15].

$$H_{\text{Ising}} = C + \sum_i h_i s_i + \sum_{i<j} J_{ij}s_is_j, \tag{10}$$

where C, h_i, and $J_{i,j}$ are constants that will define the end goal of the QA process.

D. Quantum-Assisted Restricted Boltzmann Machines:

With the QUBO model in hand and the conversion to the Ising Model determined, it is important to identify where the a quantum annealer system can make the training of RBMs more efficient. To that end, let us recall the total Hamiltonian of the system, and how its adiabatic evolution. The generic formulation of the Hamiltonian in QA systems is given by:

$$H(s) = a(s)H_D + b(s)H_P, \tag{11}$$

where s is a normalized time parameter, H_D and H_P are the driver and the problem Hamiltonians. $a(s)$ and $b(s)$ are two functions with the following pattern: when s ≈ 0, then $a(s) \approx 1$ and $b(s) \approx 0$; when s ≈ 1, then $a(s) \approx 0$ and $b(s) \approx 1$. Essentially, the process consists of a gradual transition from the driver Hamiltonian to the problem Hamiltonian.

Ideally, we would encode the target problem Hamiltonian as the QUBO problem that minimizes the energy function of the RBM, and subsequently solve it via the QA. This, however, is not possible due to NISQ quantum computers [26] being prone to noise. Be that as it may, it is still possible to optimize the model and update its parameters using PCD, while avoiding the inefficient Gibbs Sampling algorithm and replacing it with the QA. One important aspect to understand is that there is a non-zero probability that the QA process will not occur adiabatically when running in real hardware. In other

words, the system has a chance to escape the ground state and evolve towards an excited state, following the Boltzmann Distribution:

$$P(x) = \frac{1}{Z} e^{-\beta H_P(x)}, \tag{12}$$

where β is the inverse temperature parameter, linking the energy of a configuration to its sampling probability. In order to circumvent the issue, one can call the QA routine N times; with sufficient samples, one can statistically estimate the values of $\mathbf{v}_{model}, y_{model}$ and $\mathbf{h}_{model}$ using quantum hardware, without explicit computation of the intractable partition function. This process is known as Quantum Sampling (QS).

III. IMPLEMENTATIONS

A. Stone's Dataset:

The dataset for the project was provided by Brazilian fintech **Stone**, and the study revolves around identifying fraudulent transactions along with many non-fraudulent ones. The dataset covers a time span of three months with transaction data from all over Brazil. The information is anonymized in accordance with Brazilian data protection laws, so neither user information nor balance amount is disclosed. In total, there are 145 million transactions, out of which 19 thousand are fraudulent, i.e., a highly unbalanced dataset. RBM training generally suffers from such a feature, which is why dataset balancing procedures are further discussed in Section IV.B.

Furthermore, data pre-processing methodologies were introduced while consulting with Stone's anti-fraud team, making sure not to introduce unwanted bias to the dataset. These included, for the categorical data, label encoding, binary encoding, hashing, and ordinary encoding, with the most effective being the so-called One-Hot Encoding [27]. Numerical data was treated so that its distribution would follow the guidelines of a GRBM; on other words, entries were normalized leading to a respective Gaussian distribution with zero mean and unitary standard deviation. This can be done by applying the Z-Score function to all numerical columns:

$$Z = \frac{x - \mu}{\sigma}, \tag{13}$$

where x represents the numerical data and μ and σ represent the mean and the standard deviation of the column, respectively.

Finally, dataset filtering was implemented with the goal of restricting the size of the problem, an important procedure in order to enable embedding into real QA hardware. To determine the final filtered dataset, the correlation matrix of the instance features was calculated and those with the highest correlations were eliminated. Furthermore, relevance of the remaining instance features was studied using the so-called CatBoost model [28] and eliminating those with the least importance.

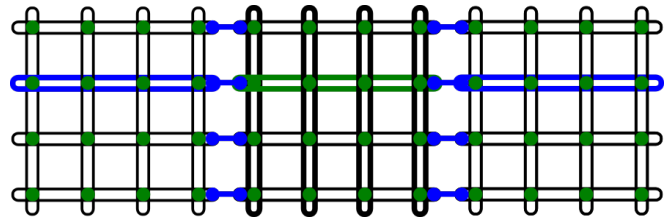

FIG. 2. Representation of a 3x3 Chimera grid, a simplified model of the D-Wave 2000Q quantum annealer. The loops represent the qubits, the blue dots represent the external couplings (which allow for the conection of two unit cells), and the internal couplings are represented by green circles (allowing the conection of one or more qubits inside the unitary cell). Image source: D-Wave's Documentation

B. D-Wave's Functionality and Embedding:

D-Wave's Hamiltonian is given by:

$$\begin{aligned} \mathcal{H}_{\text{Ising}} = & \frac{A(s)}{2} \left(\sum_i \hat{\sigma}_x^{(i)} \right) \\ & + \frac{B(s)}{2} \left(\sum_i h_i \hat{\sigma}_z^{(i)} + \sum_{i>j} J_{ij} \hat{\sigma}_z^{(i)} \hat{\sigma}_z^{(j)}, \right) \end{aligned} \tag{14}$$

where $A(s)$ is defined as the transverse energy and $B(s)$ is known as the energy applied to the problem Hamiltonian. Their functionality is essentially the same as in eq. 11: for $s \approx 0$ then $A(s) >> B(s)$ and for $s \approx 1$ then $B(s) >> A(s)$.

D-Wave's computers consist of three different topologies: Chimera, Pegasus, and Zephyr. They all work following the same principles, but can achieve different connectivity between physical qubits. The structure is composed of unit cells, a block of interconnected qubits that repeats itself, forming a grid. Different unit cells are connected to each other via couplers, allowing qubits to interact with neighboring qubits and, consequently, neighboring cells. A simplified representation of grids, cells, and qubits is depicted in Fig. 2.

For an ideal problem embedding, each hidden and visible unit of an RBM would be mapped to a single qubit. The word 'ideal' is used here since, in order to perform QA, D-Wave's systems need to form a closed loop within a unit cell [29]. That is not always possible, and when it isn't, one needs to make use of a process called *minor embedding*, which consists of chaining various qubits together to represent a more robust logical qubit. When dealing with high dimensional data, one runs into the limitation of the number of qubits in the system. This made the minor embedding process the primary bottleneck, which was overcome during the research project by restricting the dataset features as discussed in the previous Section.

C. Simulated Annealing:

An intermediate step between the strictly classical training procedure to the quantum-assisted one is the use of Simulated Annealing (SA), a feature offered by D-Wave's simulators [29]. Although this method does not take advantage of the full features of quantum annealing, its implementation is more accessible: minor embedding becomes less restrictive and the size of the RBM (number of visible and hidden units) can reach higher values than for the real hardware implementation.

D-Wave implements SA by discretizing time in different steps called *sweeps*. In each sweep, the individual qubits are updated in accordance with a transition probability derived from a Path Integral Monte Carlo (PIMC) estimation [30] in order to simulate and approximate the effects of quantum fluctuations. At the end of the process, the system should approximate a Boltzmann distribution as described in eq. 12.

D. QARBoM.jl:

In order to help with the creation, training, and optimization of hiperparameters of the RBMs, Ripper et al. [16] created `QARBoM.jl`, an open-source Julia programming language package that provides a one-stop shop of comparative training of RBMs. It provides an easy and useful way to create and train RBMs, whether using strictly classical methods or quantum-assisted ones. `QARBoM.jl` also provides a very robust backbone when dealing with QUBO problems because of its integration with the *QUBO.jl* package [31], which converts various optimization problems to QUBO. `QARBoM.jl` also directly integrates with yet another Julia package, `DWave.jl` [31], providing seamless integration with D-Wave's systems.

IV. EXPERIMENTS

This Section presents the comparative analysis between different methodologies for training RBMCs towards fraud detection based on experimental results. The three chosen methodologies include: strictly classical PCD; simulated-annealing-assisted PCD; quantum-annealing-assisted PCD. The selected figures of merit for comparison are Accuracy, Precision, Recall and F1-Score, which are standard metrics for classifiers [32]. It should be mentioned that the testing procedure consisted of using the same database described in Section III-A for all three cases, always with 50 epochs of training.

A. Dataset Balancing:

As previously discussed, the dataset is highly unbalanced, to which RBMCs are inherently sensitive. Consequently, four complementary strategies were employed to alleviate the adverse impact of this imbalance: oversampling, undersampling, oversampling followed by undersampling (and vice versa), and manual undersampling.

Oversampling consists on the usage of the SMOTE algorithm [33]. Unfortunately, it provided less than ideal results. Since the minority class was significantly small, this caused the model to learn predominantly from synthetic rather than real samples, thus producing an undesirable effect, leading to frequent misclassifications. Undersampling consists of implementing the well-known Tomek Links algorithm [34]. As with oversampling, results were far from ideal, as the minority class was extremely small. This caused the dataset to be reduced to a few hundred lines, rendering it nearly useless, as there was little to no data left for the model. Oversampling followed by Undersampling suffered from the same aforementioned limitations. The minority class was simply too small to allow for a reconstruction that enabled the model to be trained primarily on real data. Reversing the order of the approaches was also tested but led to the same issue.

Manual Undersampling was ultimately required due to the length of the dataset and a straightforward, but effective, procedure was implemented: given all the fraudulent instances, the same number of non-fraudulent ones was randomly selected and combined. This resulted in a dataset of 39 thousand transactions with a 50-50 (unbiased) split. Manual Undersampling exhibited the best training features since it only uses non-synthetic data: the model can learn to detect fraudulent transactions in a more general environment. For the evaluation of the classification metrics, the dataset was divided into training and test sets, with a 80%-20% split, respectively. All the results shown in the later sections of this chapter are metrics in relation to the test set.

B. Hyperparameter Search:

Hyperparameter search consisted of finding the best possible combination of hyperparameters for the three tested training methodologies. The first hyperparameter analyzed was the number of hidden units in each RBMC. An excessively large number of units can lead the model to overfit, while too few units may result in underfitting. The second hyperparameter examined was the learning rate, considering its initial and final values as well as the decay schedule that governs its reduction throughout training. An extensive grid search was conducted to optimize these hyperparameters, aiming to identify the configuration that maximized both recall and precision. The best hyperparameter combinations found for each RBM can be found in Figure 3.

For the strictly classical training methodology, results revealed that 200 hidden units were ideal, with an exponentially decreasing learning rate reaching zero by the end of the training procedure. For simulated-annealing-

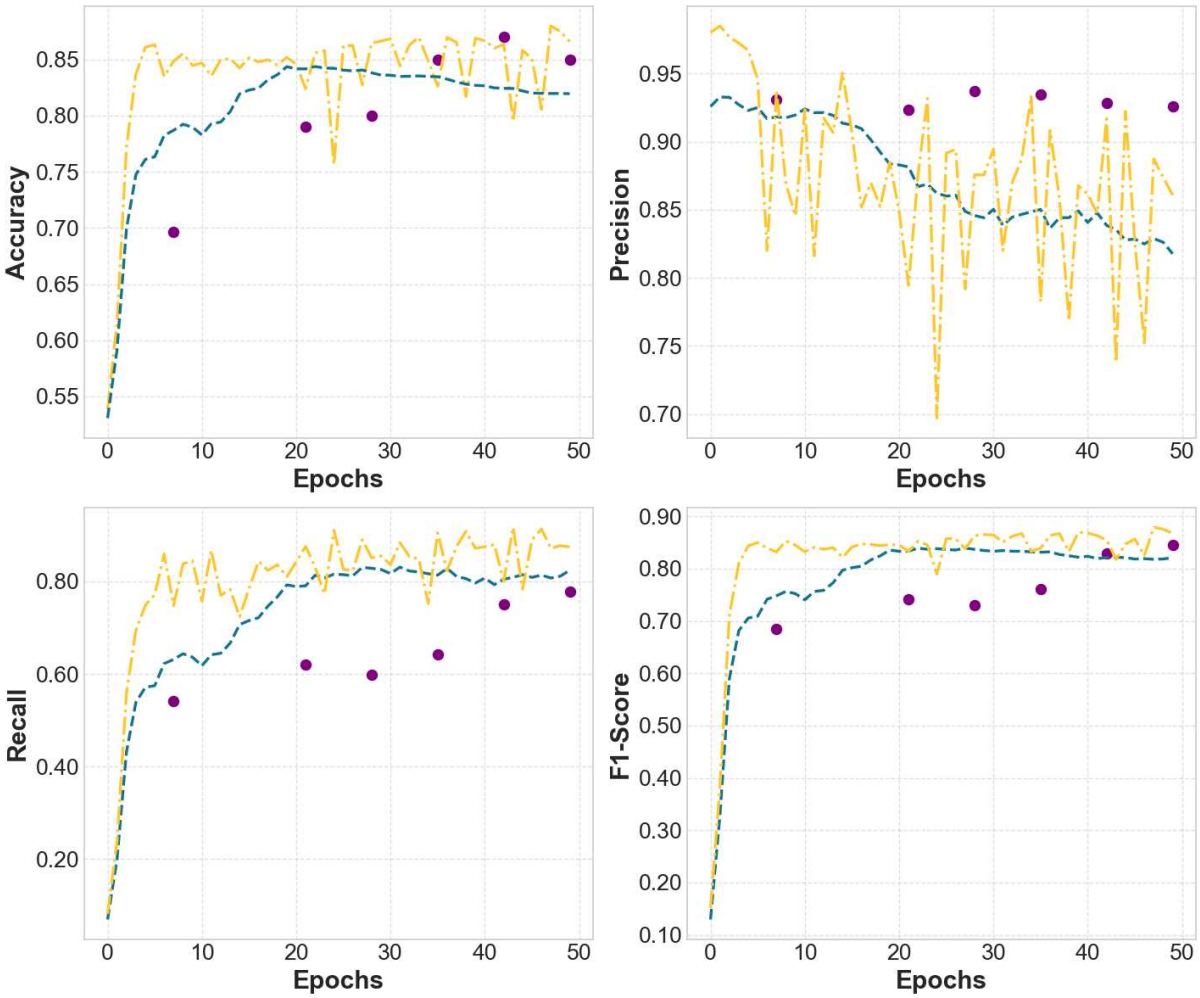

FIG. 3. Comparison between the three training methods: strictly-classical PCD (blue), simulated-annealing-assisted training (yellow), and quantum-annealing-assisted training (Purple). Training was executed in 50 epochs; for each epoch, *Accuracy*, *Precision*, *Recall*, and *F1-Score* were calculated as comparative metrics. Due to limited access to D-Wave's system, only certain values were available in the quantum-annealing-assisted training; they are plotted as purple dots in the figure. The optimum configuration achieved by each training method corresponds to epoch 25, 34, and 49, respectively.

assisted training, results revealed that 65 hidden units were ideal with the same learning rate as for the classical one. For quantum-annealing-assisted, the best number of hidden units was the same as for the simulated case: 65. The learning rate, on the other hand, differed significantly: instead of an exponential decay from an initial value towards zero, the learning rate was defined as a smooth and controllable exponential decay function, ensuring gradual convergence while avoiding premature stagnation:

$$\mathrm{LR}(\eta_0, \lambda, \eta_f) = \eta_0 \cdot \exp(-\lambda \cdot (t-1)) + \eta_f, \qquad (15)$$

where $\eta_0 + \eta_f$ represents the starting value, η_f the final value of the decay, λ the speed in which the decay occurs, and, finally, the current epoch t.

C. Results:

A noteworthy distinction between strictly-classical and – either simulated or real-hardware – quantum-assisted training arises with respect to batch size. In classical training, larger batch sizes are typically beneficial: in our experiments, batches of 512 samples were used. These batches are fed into the training model instead of the entire dataset and translate to heavier or lighter training sessions [24]. For the different flavors of quantum-assisted training, however, the opposite tends to hold, with smaller batch sizes yielding superior performance.

Simulated-annealing-assisted training results were obtained by training RBMs optimized according to the aforementioned hyperparameter search methods and using D-Wave's simulators. As with the classical model, the initial results were not promising, indicating the need

Method	Accuracy	Precision	Recall	F1-Score	Total Time
PCD	84%	86%	81%	83%	2 minutes
SAS	85%	94%	82%	88%	4 hours
QS	85%	91%	79%	84%	1.8 hours

TABLE I. Comparison of performance metrics at the optimum training epoch for each model, evaluated over a 50-epoch training process.

for dedicated hyperparameter tuning tailored specifically for this procedure. Metric evolution throughout different epochs exhibit significant oscillations, which is likely associated to the limited number of sweeps during the simulation. A larger number of sweeps translates into impractical run times, exceeding hours per epoch, and had to be constrained during the comparative analysis.

Quantum-annealing-assisted training results were obtained by training RBMs using D-Wave's quantum processors, following the same hyperparameter search optimization procedure described above. Surprisingly, both learning rates had to be significantly reduced compared to those used for the simulated-annealing-assisted training; using the same learning rates as in the previous methods caused the model to diverge. This behavior may stem from the fact that parameter updates derived from quantum-generated samples exhibit different statistical properties from those obtained through simulated sampling.

Real quantum sampling – and all quantum annealing implementations used in this project – was made possible through the Leap Program [35], which provides cloud-based access to D-Wave's quantum processors. Quantum annealing took approximately 1 hour and 50 minutes to complete the 50-epoch routine. The final training configurations are extracted from the highest ranking epoch with respect to the F1-score metric (the most relevant to the fraud detection problem according to Stone's anti-fraud team). The results are summarized in Table 1.

D. Comparison:

Fig. 3 presents the four different metrics evolution throughout the epochs for the three compared methodologies. As one can see, all models were able to achieve similar metrics, with simulated-annealing-assisted training reaching slightly higher values across all metrics, even considering its less consistent evolution (due to the limited number of sweeps, as discussed before). It is noteworthy that, even though the simulated- as well as quantum-annealing-assisted training methodologies produced similar performances with respect to the strictly classic one while running on a much lighter configuration. This is evidenced by the 55 visible, 200 hidden, 512 samples of the latter and 55 visible, 65 hidden, and 32 samples of the former two. This comparative result evidences the advantage introduced even by NISQ era platforms.

Another important figure of merit for comparison is training time, particularly given the substantial discrepancy observed across the three approaches. The fastest model, by a wide margin, was the classical one, requiring only two to three minutes to complete training. The slowest model, without question, was the simulated-annealing-assisted training method, taking approximately four hours to train. This difference in runtime can be attributed to the combination of a small batch size and the use of a Windows 11 machine. In particular, Python calls invoked from Julia incur substantial overhead on this operating system, significantly increasing execution time. A silver lining, however, is that despite being the slowest method overall, it reaches optimal performance relatively quickly, suggesting that a 50-epoch training routine was largely excessive for this case. Quantum Sampling was neither as fast as the strictly classical method nor as slow as simulated annealing one, but its training time still fell on the slower end of the spectrum.

V. CONCLUSION

In this study, a detailed investigation was carried out comparing the performance of different RBM models, including classically trained RBMs and quantum-assisted RBM models trained using both simulated and quantum annealing on real hardware. The results indicate that, for this specific problem, methods that deviate from the strictly classical methods already exhibit better performance, which can be extrapolated for more robust platforms than the current NISQ era ones. We verified that performance and runtime characteristics offer a compromise relationship: higher performance was associated with longer runtime and vice-versa. Whether this advantage is universal or specific to the characteristics of this dataset remains an open question and requires further study before a definitive conclusion can be drawn. Future work will explore new approaches to quantum-assisted machine learning, such as the quantum kernel method [36] for support vector machine training, as well as methods to further enhance the current execution of quantum-assisted training in, e.g., gate-based platforms using the Quantum Approximate Optimization Algorithm (QAOA) framework [37].

ACKNOWLEDGMENTS

This work was supported by Stone Instituição de Pagamento SA (funding number 2551.00.00) and CNPq (grant number 409596/2022-1). The group would like to extend its sincere gratitude to M. Ruas, from the Stone Anti-Fraud Team, whose assistance was instrumental to the completion of this work.

We also wish to express our most heartfelt thanks to the members of the Interdisciplinary Nucleus for Quan-

tum Technologies (NITeQ) team of PUC-Rio, A. Wanick, B. Wolf, C. Nascimento, P. Ripper and M. dos Magos, for their unwavering support and fruitful discussions throughout this project.

[1] R. P. Feynman, Simulating physics with computers, in *Feynman and computation* (cRc Press, 2018) pp. 133–153.

[2] P. W. Shor, Algorithms for quantum computation: discrete logarithms and factoring, in *Proceedings 35th annual symposium on foundations of computer science* (Ieee, 1994) pp. 124–134.

[3] N.-C. Chiu, E. C. Trapp, J. Guo, M. H. Abobeih, L. M. Stewart, S. Hollerith, P. L. Stroganov, M. Kalinowski, A. A. Geim, S. J. Evered, *et al.*, Continuous operation of a coherent 3,000-qubit system, Nature , 1 (2025).

[4] H.-S. Zhong, H. Wang, Y.-H. Deng, H. Chen, L.-Z. Peng, Y.-H. Luo, J. Qin, D. Wu, X. Ding, Y. Hu, *et al.*, Quantum computational advantage using photons, Science **370**, 1460 (2020).

[5] Quantum error correction below the surface code threshold, Nature **638**, 920 (2025).

[6] D. Deutsch, Quantum theory, the church–turing principle and the universal quantum computer, Proceedings of the Royal Society of London. A. Mathematical and Physical Sciences **400**, 97 (1985).

[7] E. Farhi, J. Goldstone, S. Gutmann, and M. Sipser, Quantum computation by adiabatic evolution, arXiv preprint quant-ph/0001106 (2000).

[8] A. Y. Kitaev, Fault-tolerant quantum computation by anyons, Annals of physics **303**, 2 (2003).

[9] S. Braunstein and A. Pati, Quantum computation with continuous variables (2003).

[10] A. S. Koshikawa, M. Ohzeki, T. Kadowaki, and K. Tanaka, Benchmark test of black-box optimization using d-wave quantum annealer, Journal of the Physical Society of Japan **90**, 064001 (2021).

[11] C. Papalitsas, T. Andronikos, K. Giannakis, G. Theocharopoulou, and S. Fanarioti, A qubo model for the traveling salesman problem with time windows, Algorithms **12**, 224 (2019).

[12] T. Kato, On the adiabatic theorem of quantum mechanics, Journal of the Physical Society of Japan **5**, 435 (1950).

[13] G. E. Hinton, S. Osindero, and Y.-W. Teh, A fast learning algorithm for deep belief nets, Neural computation **18**, 1527 (2006).

[14] S. H. Adachi and M. P. Henderson, Application of quantum annealing to training of deep neural networks, arXiv preprint arXiv:1510.06356 (2015).

[15] A. Ajagekar and F. You, Quantum computing assisted deep learning for fault detection and diagnosis in industrial process systems, Computers & Chemical Engineering **143**, 107119 (2020).

[16] P. Ripper and J. Neto, Niteq/qarbom.jl: v0.0.1 (2025).

[17] H. Larochelle, M. Mandel, R. Pascanu, and Y. Bengio, Learning algorithms for the classification restricted boltzmann machine, The Journal of Machine Learning Research **13**, 643 (2012).

[18] G. E. Hinton, Training products of experts by minimizing contrastive divergence, Neural computation **14**, 1771 (2002).

[19] R. Liao, S. Kornblith, M. Ren, D. J. Fleet, and G. Hinton, Gaussian-bernoulli rbms without tears, arXiv preprint arXiv:2210.10318 (2022).

[20] H. Robbins and S. Monro, A stochastic approximation method, The annals of mathematical statistics , 400 (1951).

[21] G. E. Hinton, A practical guide to training restricted boltzmann machines, in *Neural Networks: Tricks of the Trade: Second Edition* (Springer, 2012) pp. 599–619.

[22] T. Tieleman, Training restricted boltzmann machines using approximations to the likelihood gradient, in *Proceedings of the 25th international conference on Machine learning* (2008) pp. 1064–1071.

[23] S. Geman and D. Geman, Stochastic relaxation, gibbs distributions, and the bayesian restoration of images, IEEE Transactions on Pattern Analysis and Machine Intelligence **PAMI-6**, 721 (1984).

[24] G. E. Hinton, *A Practical Guide to Training Restricted Boltzmann Machines*, Tech. Rep. UTML TR 2010-003 (University of Toronto, 2010).

[25] A. P. Punnen, The quadratic unconstrained binary optimization problem, Springer International Publishing **10**, 978 (2022).

[26] J. Preskill, Quantum computing in the nisq era and beyond, Quantum **2**, 79 (2018).

[27] D. Ruppert, The elements of statistical learning: data mining, inference, and prediction (2004).

[28] J. T. Hancock and T. M. Khoshgoftaar, Catboost for big data: an interdisciplinary review, Journal of big data **7**, 94 (2020).

[29] V. Choi, Minor-embedding in adiabatic quantum computation: I. the parameter setting problem, Quantum Information Processing **7**, 193 (2008).

[30] D. M. Ceperley, Path integrals in the theory of condensed helium, Reviews of Modern Physics **67**, 279 (1995).

[31] P. M. Xavier, P. Ripper, T. Andrade, J. D. Garcia, N. Maculan, and D. E. B. Neira, Qubo. jl: A julia ecosystem for quadratic unconstrained binary optimization, arXiv preprint arXiv:2307.02577 (2023).

[32] D. M. Powers, Evaluation: from precision, recall and f-measure to roc, informedness, markedness and correlation, arXiv preprint arXiv:2010.16061 (2020).

[33] N. V. Chawla, K. W. Bowyer, L. O. Hall, and W. P. Kegelmeyer, Smote: synthetic minority over-sampling technique, Journal of artificial intelligence research **16**, 321 (2002).

[34] I. Tomek, Two modifications of cnn, IEEE Transactions on Systems, Man, and Cybernetics **SMC-6**, 769 (1976).

[35] D-Wave Quantum Inc., The leap™ quantum cloud service (2024), accessed: 2025-12-08.

[36] P. Rebentrost, M. Mohseni, and S. Lloyd, Quantum support vector machine for big data classification, arXiv preprint arXiv:1307.0471 (2013).

[37] P. Díez-Valle, D. Porras, and J. J. García-Ripoll, Quantum approximate optimization algorithm pseudo-boltzmann states, Physical review letters **130**, 050601 (2023).